Department of Computer Science and Engineering

Babu Banarasi Das University, Lucknow



School of Engineering

DISTRIBUTED SYSTEM

Lab File

(NCS4751)

Session: 2026-2027

Fourth Year (7th Semester)

Submitted by :

Jaidev Yadav

(1230432336)

B.Tech CSE 4F

Submitted to:

Ms. Divya Agrahari

(Assistant Professor)

Department of CSE

INDEX

S. No.	List of Experiments	CO	Remarks / Sign
1	Simulate the functioning of Lamport's Logical Clock in 'C'.	CO2	
2	Simulate the Distributed Mutual Exclusion in 'C'.	CO1	
3	Implement a Distributed Chat Server using TCP Sockets in 'C'.	CO1	
4	Implement RPC mechanism for a file transfer across a network in 'C'.	CO2	
5	Implement 'Java RMI' mechanism for accessing methods of remote systems.	CO3	
6	Simulate Balanced Sliding Window Protocol in 'C'.	CO3	
7	Implement CORBA mechanism by using 'C++' program at one end and 'Java' program on the other.	CO4	
8	Program to implement edge chasing distributed deadlock detection algorithm.	CO3	
9	Program to implement locking algorithm.	CO4	
10	Program to implement termination detection.	CO4	

PROGRAM - 1

Aim:

To simulate the functioning of Lamport's Logical Clock using the C programming language.

Code:

```
#include <stdio.h>

int main()
{
    int p1[10], p2[10];
    int n1, n2, i;

    printf("Enter number of events in Process P1: ");
    scanf("%d", &n1);
    printf("Enter number of events in Process P2: ");
    scanf("%d", &n2);

    // Initialize logical clocks
    for (i = 0; i < n1; i++)
        p1[i] = i + 1;
    for (i = 0; i < n2; i++)
        p2[i] = i + 1;

    int send, receive;
    printf("\nEnter sending event number in P1: ");
    scanf("%d", &send);
    printf("Enter receiving event number in P2: ");
    scanf("%d", &receive);

    if (p2[receive - 1] <= p1[send - 1])
    {
        p2[receive - 1] = p1[send - 1] + 1;
        for (i = receive; i < n2; i++)
            p2[i] = p2[i - 1] + 1;
    }

    printf("\nLogical Clock of Process P1:\n");
    for (i = 0; i < n1; i++)
        printf("Event %d = %d\n", i + 1, p1[i]);

    printf("\nLogical Clock of Process P2:\n");
    for (i = 0; i < n2; i++)
        printf("Event %d = %d\n", i + 1, p2[i]);

    return 0;
}
```

Output:

```
Enter number of events in Process P1: 5
Enter number of events in Process P2: 5

Enter sending event number in P1: 3
Enter receiving event number in P2: 2

Logical Clock of Process P1:
Event 1 = 1
Event 2 = 2
Event 3 = 3
Event 4 = 4
Event 5 = 5

Logical Clock of Process P2:
```

```
Event 1 = 1
Event 2 = 4
Event 3 = 5
Event 4 = 6
Event 5 = 7
```

PROGRAM - 2

Aim:

To simulate the Distributed Mutual Exclusion algorithm using C.

Code:

```
#include <stdio.h>

int main()
{
    int n, i;
    int cs = 0;   // Critical Section status (0 = Free, 1 = Busy)

    printf("Distributed Mutual Exclusion Simulation\n");
    printf("--------------------------------------\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        printf("\nProcess P%d requests Critical Section.\n", i);

        if (cs == 0)
        {
            cs = 1;
            printf("Critical Section is FREE.\n");
            printf("Process P%d ENTERS Critical Section.\n", i);
            printf("Process P%d is EXECUTING...\n", i);
            printf("Process P%d LEAVES Critical Section.\n", i);
            cs = 0;
            printf("Critical Section is now FREE.\n");
        }
        else
        {
            printf("Process P%d is WAITING.\n", i);
        }
    }

    printf("\nAll Processes Executed Successfully.\n");
    return 0;
}
```

Output:

```
Distributed Mutual Exclusion Simulation
--------------------------------------
Enter number of processes: 4

Process P1 requests Critical Section.
Critical Section is FREE.
Process P1 ENTERS Critical Section.
Process P1 is EXECUTING...
Process P1 LEAVES Critical Section.
Critical Section is now FREE.

Process P2 requests Critical Section.
Critical Section is FREE.
Process P2 ENTERS Critical Section.
Process P2 is EXECUTING...
Process P2 LEAVES Critical Section.
Critical Section is now FREE.

Process P3 requests Critical Section.
Critical Section is FREE.
Process P3 ENTERS Critical Section.
```

```
Process P3 is EXECUTING...
Process P3 LEAVES Critical Section.
Critical Section is now FREE.

Process P4 requests Critical Section.
Critical Section is FREE.
Process P4 ENTERS Critical Section.
Process P4 is EXECUTING...
Process P4 LEAVES Critical Section.
Critical Section is now FREE.

All Processes Executed Successfully.
```

PROGRAM - 3

Aim:

To implement a simple Distributed Chat Server using TCP Sockets in C for communication between a server and a client.

Code:

Server Program (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    char *message = "Hello Client";

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);

    printf("Waiting for client...\n");

    new_socket = accept(server_fd, (struct sockaddr *)&address,
                        (socklen_t *)&addrlen);

    read(new_socket, buffer, 1024);
    printf("Client: %s\n", buffer);

    send(new_socket, message, strlen(message), 0);
    printf("Reply Sent.\n");

    close(new_socket);
    close(server_fd);
    return 0;
}
```

Client Program (client.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
```

```
{
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    char message[100];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));

    printf("Enter Message : ");
    scanf("%s", message);

    send(sock, message, strlen(message), 0);
    read(sock, buffer, 1024);

    printf("Server: %s\n", buffer);

    close(sock);
    return 0;
}
```

Output:

Server (Terminal 1)

```
Waiting for client...
Client: Hello
Reply Sent.
```

Client (Terminal 2)

```
Enter Message : Hello
Server: Hello Client
```

PROGRAM - 4

Aim:

To implement a Remote Procedure Call (RPC) mechanism for transferring a file across a network using C.

Code:

Server Program (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define SIZE 1024

int main()
{
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char filename[100] = {0}, buffer[SIZE];
    FILE *fp;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 5);

    printf("Waiting for Client...\n");

    new_socket = accept(server_fd, (struct sockaddr *)&address,
                        (socklen_t *)&addrlen);

    recv(new_socket, filename, sizeof(filename), 0);
    printf("Client Requested: %s\n", filename);

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        strcpy(buffer, "File Not Found");
        send(new_socket, buffer, strlen(buffer), 0);
    }
    else
    {
        while (fgets(buffer, SIZE, fp) != NULL)
        {
            send(new_socket, buffer, strlen(buffer), 0);
        }
        fclose(fp);
        printf("File Sent Successfully.\n");
    }

    close(new_socket);
    close(server_fd);
```

```
    return 0;
}
```

Client Program (client.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define SIZE 1024

int main()
{
    int sock, n;
    struct sockaddr_in serv_addr;
    char filename[100], buffer[SIZE];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));

    printf("Enter File Name: ");
    scanf("%s", filename);

    send(sock, filename, strlen(filename) + 1, 0);

    printf("Receiving File...\n");
    FILE *fp = fopen("ReceivedFile.txt", "w");

    while ((n = recv(sock, buffer, SIZE, 0)) > 0)
    {
        fwrite(buffer, 1, n, fp);
    }

    fclose(fp);
    printf("File Received Successfully.\n");

    close(sock);
    return 0;
}
```

Output:

Server (Terminal 1)

```
Waiting for Client...
Client Requested: sample.txt
File Sent Successfully.
```

Client (Terminal 2)

```
Enter File Name: sample.txt
Receiving File...
File Received Successfully.
```

PROGRAM - 5

Aim:

To implement the Java RMI (Remote Method Invocation) mechanism for accessing methods of a remote system.

Code:

Remote Interface (RemoteCalculator.java)

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface RemoteCalculator extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

Remote Implementation (RemoteCalculatorImpl.java)

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class RemoteCalculatorImpl extends UnicastRemoteObject
        implements RemoteCalculator {

    public RemoteCalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

RMI Server (RMIServer.java)

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIServer {
    public static void main(String[] args) {
        try {
            RemoteCalculatorImpl obj = new RemoteCalculatorImpl();
            Registry registry = LocateRegistry.createRegistry(1099);
            registry.rebind("CalculatorService", obj);

            System.out.println("RMI Server is running...");
            System.out.println("Remote object registered as CalculatorService");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

RMI Client (RMIClient.java)

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIClient {
    public static void main(String[] args) {
        try {
```

```
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            RemoteCalculator calculator =
                (RemoteCalculator) registry.lookup("CalculatorService");

            int result = calculator.add(20, 30);
            System.out.println("Result received from remote system: " + result);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Server

```
RMI Server is running...
Remote object registered as CalculatorService
```

Client

```
Result received from remote system: 50
```

PROGRAM - 6

Aim:

To simulate the Balanced Sliding Window Protocol using C.

Code:

```
#include <stdio.h>

int main()
{
    int window_size, num_frames, frames[51], i;

    printf("Enter window size: ");
    scanf("%d", &window_size);

    printf("\nEnter number of frames to transmit: ");
    scanf("%d", &num_frames);

    printf("\nEnter %d frames: ", num_frames);
    for (i = 1; i <= num_frames; i++)
    {
        scanf("%d", &frames[i]);
    }

    printf("\nWith sliding window protocol, the frames will be sent as follows:\n");
    printf("After sending %d frames, sender waits for acknowledgment.\n\n",
           window_size);

    for (i = 1; i <= num_frames; i++)
    {
        if (i % window_size == 0)
        {
            printf("%d\n", frames[i]);
            printf("Acknowledgment received for above frames.\n\n");
        }
        else
        {
            printf("%d ", frames[i]);
        }
    }

    if (num_frames % window_size != 0)
    {
        printf("\nAcknowledgment received for above frames.\n");
    }

    return 0;
}
```

Output:

```
Enter window size: 3

Enter number of frames to transmit: 7

Enter 7 frames: 1 2 3 4 5 6 7

With sliding window protocol, the frames will be sent as follows:
After sending 3 frames, sender waits for acknowledgment.

1 2 3
Acknowledgment received for above frames.

4 5 6
```

Acknowledgment received for above frames.

7
Acknowledgment received for above frames.

PROGRAM - 7

Aim:

To demonstrate CORBA (Common Object Request Broker Architecture) communication between a C++ server and a Java client using a shared IDL interface.

Code:

Calculator.idl

```
module Demo {
    interface Calculator {
        long add(in long a, in long b);
    };
};
```

C++ Server (server.cpp)

```
#include <iostream>
#include <fstream>
#include <omniORB4/CORBA.h>
#include "Calculator.hh"

using namespace std;

class CalculatorImpl : public POA_Demo::Calculator {
public:
    CORBA::Long add(CORBA::Long a, CORBA::Long b) override {
        cout << "Request received from Java client." << endl;
        return a + b;
    }
};

int main(int argc, char* argv[]) {
    try {
        // Initialize the ORB
        CORBA::ORB_var orb = CORBA::ORB_init(argc, argv);

        // Get the Root POA
        CORBA::Object_var obj = orb->resolve_initial_references("RootPOA");
        PortableServer::POA_var poa = PortableServer::POA::_narrow(obj);

        // Create the C++ servant
        CalculatorImpl servant;

        // Activate the servant
        PortableServer::ObjectId_var id = poa->activate_object(&servant);

        // Obtain the CORBA object reference
        CORBA::Object_var ref = poa->servant_to_reference(&servant);

        // Convert the reference into an IOR string
        CORBA::String_var ior = orb->object_to_string(ref);

        // Save the IOR for the Java client
        ofstream file("calculator.ior");
        file << ior.in();
        file.close();

        cout << "C++ CORBA Server is running." << endl;
        cout << "IOR saved in calculator.ior" << endl;
```

```
        // Start processing requests
        PortableServer::POAManager_var manager = poa->the_POAManager();
        manager->activate();
        orb->run();
        orb->destroy();
    }
    catch (const CORBA::Exception& ex) {
        cerr << "CORBA exception: " << ex._name() << endl;
        return 1;
    }
    return 0;
}
```

Java Client (Client.java)

```
import org.omg.CORBA.ORB;
import Demo.Calculator;
import Demo.CalculatorHelper;
import java.nio.file.Files;
import java.nio.file.Paths;

public class Client {
    public static void main(String[] args) {
        try {
            // Initialize the Java ORB
            ORB orb = ORB.init(args, null);

            // Read the server's IOR
            String ior = new String(
                Files.readAllBytes(Paths.get("calculator.ior"))
            ).trim();

            // Convert IOR into a CORBA object
            org.omg.CORBA.Object obj = orb.string_to_object(ior);

            // Obtain the Calculator interface
            Calculator calculator = CalculatorHelper.narrow(obj);

            // Invoke the remote C++ method
            int result = calculator.add(10, 20);
            System.out.println("Result received from C++ server: " + result);

            orb.destroy();
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

C++ server terminal

```
C++ CORBA Server is running.
IOR saved in calculator.ior
Request received from Java client.
```

Java client terminal

```
Result received from C++ server: 30
```

PROGRAM - 8

Aim:

To implement the Edge Chasing Distributed Deadlock Detection algorithm in Java.

Code:

```
import java.util.*;

class Process {
    int id;
    List<Integer> waitingFor;

    Process(int id) {
        this.id = id;
        waitingFor = new ArrayList<>();
    }
}

public class EdgeChasingDeadlock {
    static Process[] processes;
    static boolean deadlock = false;

    static void edgeChasing(int initiator, int current, boolean[] visited) {
        visited[current] = true;

        for (int next : processes[current].waitingFor) {
            System.out.println("Probe: (" + initiator + " -> " + current
                               + " -> " + next + ")");

            if (next == initiator) {
                deadlock = true;
                return;
            }

            if (!visited[next]) {
                edgeChasing(initiator, next, visited);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of processes: ");
        int n = sc.nextInt();

        processes = new Process[n];
        for (int i = 0; i < n; i++)
            processes[i] = new Process(i);

        System.out.println("Enter waiting dependency matrix (0/1):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int x = sc.nextInt();
                if (x == 1)
                    processes[i].waitingFor.add(j);
            }
        }

        System.out.print("Enter initiator process: ");
        int initiator = sc.nextInt();

        boolean[] visited = new boolean[n];
```

```
        edgeChasing(initiator, initiator, visited);

        if (deadlock)
            System.out.println("\nDeadlock Detected!");
        else
            System.out.println("\nNo Deadlock Detected.");

        sc.close();
    }
}
```

Output:

Case 1: Deadlock (cycle 0 -> 1 -> 2 -> 3 -> 0)

```
Enter number of processes: 4
Enter waiting dependency matrix (0/1):
0 1 0 0
0 0 1 0
0 0 0 1
1 0 0 0
Enter initiator process: 0
Probe: (0 -> 0 -> 1)
Probe: (0 -> 1 -> 2)
Probe: (0 -> 2 -> 3)
Probe: (0 -> 3 -> 0)

Deadlock Detected!
```

Case 2: No deadlock

```
Enter number of processes: 3
Enter waiting dependency matrix (0/1):
0 1 0
0 0 1
0 0 0
Enter initiator process: 0
Probe: (0 -> 0 -> 1)
Probe: (0 -> 1 -> 2)

No Deadlock Detected.
```